

Writing a custom action

Week 6 Lecture 1: Browser Use: How LLM Agents Drive the Web

Summer 2026

What we will cover

- The Tools registry: how built-in actions and custom actions share the same structure
- The `@tools.action()` decorator: registering a handler in three lines
- Typed parameters: plain signatures, Pydantic models, and injected dependencies
- `ActionResult`: what your action returns and how the loop uses it
- `allowed_domains`: giving an action a domain scope

By the end, you will be able to write, register, and test a custom action that the agent can call.

From the registry to your function

Every action is a registered entry

The Tools registry maps action names to handler functions and their Pydantic parameter models.

- Built-in actions (click, type, navigate, done) are pre-registered on the default Tools instance
- Custom actions add new entries to the same registry
- The registry serializes each action's name, description, and parameter schema into the system prompt
- The LLM picks from this list: it never calls Python directly

When you write a custom action, you are adding a new row to the table the LLM reads.

The @tools.action() decorator

Create a Tools instance, decorate a function, pass tools to Agent.

```
from browser_use import Agent, Tools, ActionResult, ChatOpenAI

tools = Tools()

@tools.action(description='Ask the user a question and return their answer')
async def ask_human(question: str) -> ActionResult:
    answer = input(f'{question} > ')
    return ActionResult(extracted_content=f'Human answered: {answer}')

agent = Agent(task='...', llm=ChatOpenAI(model='gpt-4.1-mini'), tools=tools)
```

The description is what the LLM reads to decide when to call this action.

Source: Browser Use docs: Custom Functions (browser-use team, 2026)

Typed parameters

Two ways to declare parameters

Plain function signature: each parameter becomes a field in the generated schema.

```
@tools.action(description='Save a note to disk')
def save_note(filename: str, content: str) -> str:
    with open(filename, 'w') as f:
        f.write(content)
    return f'Saved {len(content)} chars to {filename}'
```

Pydantic param_model: pass the whole model as one argument. Use this when the schema is complex or you want Field-level descriptions.

```
from pydantic import BaseModel, Field

class NoteParams(BaseModel):
    filename: str = Field(description='Target file path, e.g. "notes/todo.txt"')
    content: str = Field(description='Text to write')
```

```
@tools.action(description='Save a note to disk', param_model=NoteParams)
```

Injected dependencies

Parameters named after framework objects are filled automatically.

```
from browser_use import BrowserSession

@tools.action(description='Return the text of all headings on the current page')
async def extract_headings(browser_session: BrowserSession) -> ActionResult:
    page = await browser_session.must_get_current_page()
    elements = await page.get_elements_by_css_selector('h1, h2, h3')
    headings = [await el.get_text() for el in elements]
    text = '\n'.join(headings)
    return ActionResult(extracted_content=text)
```

Available injected names: `browser_session`, `page_extraction_llm`, `file_system`, `available_file_paths`.

Name matching is exact. `browser_session: BrowserSession` works; `bs: BrowserSession` does not.

ActionResult

What your action returns

ActionResult is a Pydantic model. The loop reads it after every action.

Field	Purpose
extracted_content	Text the agent can see next step
long_term_memory	Persisted note across many steps
is_done	True signals the task is complete
success	Only valid when is_done=True
error	String description of a failure

```
return ActionResult(  
    extracted_content='Found 3 matching rows.',  
    long_term_memory='Rows: 42, 17, 9'  
)
```

Restricting an action with allowed_domains

Domain-scoped actions

The `@tools.action()` decorator accepts an `allowed_domains` list.

```
@tools.action(
    description='Save the page title to a log file',
    allowed_domains=['news.ycombinator.com', '*.github.com']
)
async def log_page_title(browser_session: BrowserSession) -> ActionResult:
    page = await browser_session.must_get_current_page()
    title = await page.title()
    with open('titles.log', 'a') as f:
        f.write(title + '\n')
    return ActionResult(extracted_content=f'Logged: {title}')
```

On every step, the registry recalculates which actions are visible given the current URL. If the agent is on a domain not in the list, this action disappears from the LLM's prompt.

Source: Browser Use docs: Custom Functions (browser-use team, 2026)

When to write a custom action

The built-in actions already handle clicking, typing, scrolling, navigating, and extracting.

Write a custom action when:

- You need side effects outside the browser (write a file, call an API, query a database)
- The task requires domain logic the LLM cannot infer from the page alone
- You want to expose a tool only on specific domains to prevent mis-use
- The extraction needs to run specific CSS selectors or structured parsing

Do not write a custom action to replace click or type. The built-in implementations are already aware of element indices and handle edge cases.

Take-aways

1. `@tools.action()` registers a function on a Tools instance. Pass that instance to `Agent(tools=...)`.
2. Describe the action clearly: the LLM reads the description to decide when to use it.
3. Pydantic `param_model` gives you Field-level descriptions; plain signatures work for simple cases.
4. `ActionResult` carries content forward: `extracted_content` is visible next step; `long_term_memory` persists longer.
5. `allowed_domains` scopes an action to specific URLs so the LLM only sees it when relevant.

"Action filters (domains) let you limit actions available to the Agent on a step-by-step/page-by-page basis. This helps prevent the LLM from deciding to use an action that is not compatible with the current page."

What is next

- **Lecture 2 (this week):** output_model_schema for typed extraction results; sensitive_data placeholder substitution; allowed_domains on BrowserProfile
- **Section (this week):** Build an agent with one custom extraction tool and a structured output model; verify the schema holds against the real output
- **Reading (required before section):** Week 6 reading at [/c/browser-use-26su/readings/wk06](#)